



DEEP LEARNING AND NATURAL LANGUAGE PROCESSING

LECTURE 2: MLP ARCHITECTURE AND CORE CONCEPTS

Dr. Adewale Akinfaderin

SEAS 8520-D6A · Summer 2026



Table of Contents

1 Recap and Roadmap

- ▶ Recap and Roadmap
- ▶ Tensors: The Data Structure of Deep Learning
- ▶ Tensor Operations
- ▶ The Keras/TensorFlow Ecosystem
- ▶ Building an MLP: Architecture Decisions
- ▶ MNIST End-to-End
- ▶ Architecture Design Choices
- ▶ Experiments and Diagnostics

What we covered:

- $AI \supset ML \supset DL$ hierarchy
- Classical programming vs. ML paradigm
- Neurons, layers, and nonlinear activations
- Gradient-based learning (high level)
- Historical context: perceptrons to Transformers

Key takeaway:

Deep learning learns *hierarchical representations* from data through compositions of differentiable layers.

Three pillars:

1. Massive datasets
2. GPU/TPU hardware
3. Algorithmic advances



Today's Roadmap

1 Recap and Roadmap

Topics:

1. Mathematical foundations: tensors
2. Tensor operations and broadcasting
3. The Keras/TensorFlow ecosystem
4. Building your first neural network
5. MNIST digit classification (end to end)
6. MLP architecture design choices

Reading:

Chollet, Ch. 2, pp. 27–48

Homework 1: Due May 22

Learning objectives:

- Understand tensors and their operations
- Use Keras to define, compile, and train models
- Build an end-to-end MLP pipeline



Table of Contents

2 Tensors: The Data Structure of Deep Learning

- ▶ Recap and Roadmap
- ▶ Tensors: The Data Structure of Deep Learning
- ▶ Tensor Operations
- ▶ The Keras/TensorFlow Ecosystem
- ▶ Building an MLP: Architecture Decisions
- ▶ MNIST End-to-End
- ▶ Architecture Design Choices
- ▶ Experiments and Diagnostics

A **tensor** is a generalization of matrices to arbitrary dimensions (called *axes* or *ranks*).

Terminology:

- **Rank 0:** Scalar (single number)
- **Rank 1:** Vector (1D array)
- **Rank 2:** Matrix (2D array)
- **Rank 3+:** Higher-order tensor

Key attributes:

- **ndim:** number of axes (rank)
- **shape:** tuple of dimension sizes
- **dtype:** data type (`float32`, `int32`)

Every piece of data in a neural network (inputs, weights, outputs, gradients) is stored as a tensor.

A scalar contains a single number.

In NumPy: `ndim = 0`, `shape = ()`

Examples in DL:

- Learning rate $\eta = 0.001$
- Loss value $\mathcal{L} = 2.34$
- A single pixel intensity

```
1 import numpy as np
2
3 x = np.array(42)
4 print(x.ndim)      # 0
5 print(x.shape)     # ()
6 print(x.dtype)     # int64
```

Scalars are the building blocks. A vector is a sequence of scalars; a matrix is a grid of scalars.

A vector is an array of scalars. It has one axis.

Shape: $(n,)$ where n is the number of elements.

Examples in DL:

- A bias vector $\mathbf{b} \in \mathbb{R}^{128}$
- A word embedding (300 floats)
- A single flattened image (784 pixels)

```
1 x = np.array([1.0, 2.0, 3.0, 4.0])
2 print(x.ndim)      # 1
3 print(x.shape)     # (4,)
4
5 # A 5-dimensional vector
6 v = np.array([0.1, 0.3, 0.5, 0.7,
7               0.9])
8 # This is a rank-1 tensor
9 # living in 5D space
```


A matrix has two axes: rows and columns.

Shape: (rows, cols)

Examples in DL:

- Weight matrix $W \in \mathbb{R}^{128 \times 64}$
- A batch of feature vectors
- A single grayscale image (28×28)

```
1 W = np.array([[1, 2, 3],  
2               [4, 5, 6],  
3               [7, 8, 9]])  
4 print(W.ndim)      # 2  
5 print(W.shape)     # (3, 3)  
6  
7 # Weight matrix for a dense layer  
8 W = np.random.randn(784, 128)  
9 # Shape: (input_dim, output_dim)
```

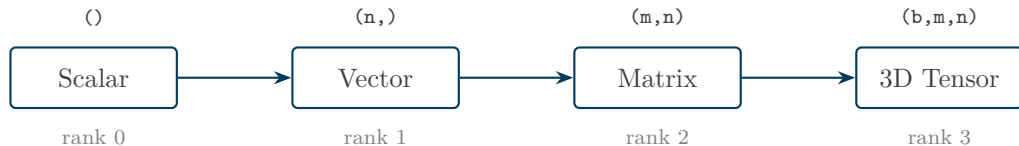
Pack matrices into an array: rank-3 tensor.

Pack rank-3 tensors: rank-4 tensor.

Common shapes in DL:

- (batch, timesteps, features):
sequences
- (batch, height, width, channels):
images
- (batch, frames, h, w, c): video

```
1 # A batch of 32 grayscale images
2 # Each image: 28x28 pixels
3 images = np.zeros((32, 28, 28))
4 print(images.ndim)      # 3
5 print(images.shape)     # (32, 28, 28)
6
7 # A batch of 32 color images
8 color = np.zeros((32, 28, 28, 3))
9 print(color.ndim)       # 4
10 print(color.shape)      # (32, 28, 28,
                           3)
```

**Rule of thumb:**

Each axis adds a “container” dimension.

A rank- n tensor is an array of rank- $(n-1)$ tensors.

In practice:

Most DL data is rank 2–5.

Rank 0–1 appear as parameters (scalars, biases).

Data Type	Shape	Example
Tabular data	(samples, features)	(10000, 12)
Grayscale images	(samples, H, W)	(60000, 28, 28)
Color images	(samples, H, W, 3)	(50000, 32, 32, 3)
Text sequences	(samples, seq_len)	(25000, 500)
Time series	(samples, timesteps, channels)	(1000, 60, 5)
Video	(samples, frames, H, W, 3)	(100, 30, 224, 224, 3)

The first axis is almost always the **batch dimension** (number of samples). This enables parallelism on GPU.

The MNIST dataset:

- 60,000 training images
- 10,000 test images
- Each image: 28×28 grayscale pixels
- Pixel values: integers in $[0, 255]$

Training tensor shape:

(60000, 28, 28) – rank-3, dtype uint8

After preprocessing:

Normalize to $[0, 1]$: divide by 255.0

Flatten to vectors: (60000, 784)

```
1 from tensorflow.keras.datasets
   import mnist
2
3 (train_images, train_labels), \
4 (test_images, test_labels) = mnist.
   load_data()
5
6 print(train_images.shape)
7 # (60000, 28, 28)
8 print(train_images.dtype)
9 # uint8
10
11 # Preprocessing
12 train_images = train_images.reshape
   (
13     (60000, 784)).astype("float32")
   / 255
```

Selecting specific elements or sub-tensors along each axis.

NumPy slicing: `tensor[start:stop]`

Examples:

- First 100 images: `train[0:100]`
- A single image: `train[42]`
- Top-left corner: `img[0:14, 0:14]`

```
1 # Select images 10 through 99
2 batch = train_images[10:100]
3 print(batch.shape) # (90, 28, 28)
4
5 # Select a 14x14 patch
6 # from each image in the batch
7 patches = batch[:, 0:14, 0:14]
8 print(patches.shape)
9 # (90, 14, 14)
10
11 # Negative indexing
12 last_10 = train_images[-10:]
```



Table of Contents

3 Tensor Operations

- ▶ Recap and Roadmap
- ▶ Tensors: The Data Structure of Deep Learning
- ▶ **Tensor Operations**
- ▶ The Keras/TensorFlow Ecosystem
- ▶ Building an MLP: Architecture Decisions
- ▶ MNIST End-to-End
- ▶ Architecture Design Choices
- ▶ Experiments and Diagnostics

Neural networks are compositions of tensor operations:

$$\hat{\mathbf{y}} = \sigma(W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 \cdot \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3)$$

Every forward pass reduces to:

1. **Element-wise operations:** activation functions, addition
2. **Tensor dot products:** matrix multiplications (linear projections)
3. **Broadcasting:** adding bias vectors to matrix outputs
4. **Reshaping:** changing tensor geometry without copying data

Understanding these operations is essential for debugging shapes, designing architectures, and reasoning about computational cost.

Applied independently to each entry of the tensor.

Examples:

- Addition: $C_{ij} = A_{ij} + B_{ij}$
- Multiplication: $C_{ij} = A_{ij} \times B_{ij}$
- ReLU: $\text{relu}(x)_{ij} = \max(0, x_{ij})$

Requirement: tensors must have the *same shape* (or be broadcastable).

```
1 a = np.array([1.0, 2.0, 3.0])
2 b = np.array([4.0, 5.0, 6.0])
3
4 # Element-wise addition
5 c = a + b # [5., 7., 9.]
6
7 # Element-wise multiply
8 d = a * b # [4., 10., 18.]
9
10 # ReLU (element-wise max)
11 x = np.array([-1, 0.5, -0.3, 2])
12 relu = np.maximum(x, 0)
13 # [0., 0.5, 0., 2.]
```

When tensors have different shapes, NumPy (and TensorFlow) can still perform element-wise operations by “broadcasting” the smaller tensor.

Rules:

1. Align shapes from the right
2. Dimensions match if equal or one is 1
3. The size-1 dimension is repeated

In neural networks:

Adding bias $\mathbf{b} \in \mathbb{R}^n$ to a batch output $X \in \mathbb{R}^{m \times n}$ uses broadcasting.

```
1 # X: (32, 128) -- batch output
2 # b: (128,)    -- bias vector
3 X = np.random.randn(32, 128)
4 b = np.random.randn(128)
5
6 # Broadcasting adds b to each row
7 Y = X + b # shape: (32, 128)
8
9 # Equivalent to:
10 # for i in range(32):
11 #     Y[i] = X[i] + b
```

Broadcasting avoids explicit loops and runs at C/CUDA speed.

$$\begin{array}{c} X: (3, 4) \\ \boxed{\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}} \end{array} + \begin{array}{c} \mathbf{b}: (4,) \\ \boxed{\begin{bmatrix} 10 & 20 & 30 & 40 \end{bmatrix}} \end{array} = \begin{array}{c} Y: (3, 4) \\ \boxed{\begin{bmatrix} 11 & 22 & 33 & 44 \\ 15 & 26 & 37 & 48 \\ 19 & 30 & 41 & 52 \end{bmatrix}} \end{array}$$

The vector **b** is conceptually replicated along axis 0 to match the matrix shape. No actual memory copy occurs.

The fundamental operation of neural networks.

For matrices $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$:

$$C = AB, \quad C_{ij} = \sum_{p=1}^k A_{ip} B_{pj}$$

Result shape: (m, n) .

Constraint: inner dimensions must match.
 $(m, \mathbf{k}) \cdot (\mathbf{k}, n) \rightarrow (m, n)$

```
1 # Dense layer forward pass
2 # x: (batch_size, input_dim)
3 # W: (input_dim, output_dim)
4 x = np.random.randn(32, 784)
5 W = np.random.randn(784, 128)
6
7 output = np.dot(x, W)
8 print(output.shape) # (32, 128)
9
10 # Computational cost:
11 # 32 * 784 * 128 = 3,211,264
12 # multiply-add operations
```

A dense layer computes:

$$\mathbf{a} = \sigma(W\mathbf{x} + \mathbf{b})$$

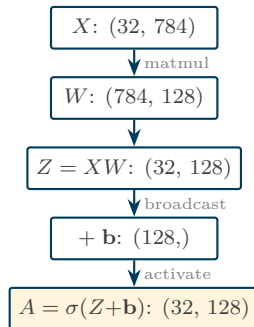
For a batch of m samples:

$$A = \sigma(XW + \mathbf{b})$$

Dimensions:

- X : $(m \times d_{\text{in}})$ – input batch
- W : $(d_{\text{in}} \times d_{\text{out}})$ – weights
- $\mathbf{b}$: $(d_{\text{out}},)$ – bias (broadcast)
- A : $(m \times d_{\text{out}})$ – output batch

Concrete example:



Rearranges tensor elements into a new shape without changing the data.

Constraint: total number of elements must be preserved.

Common uses:

- Flatten images: $(28, 28) \rightarrow (784,)$
- Add batch dim: $(784,) \rightarrow (1, 784)$
- Transpose: swap axes

```
1 # Flatten a 28x28 image
2 img = np.zeros((28, 28))
3 flat = img.reshape((784,))
4
5 # Flatten a batch
6 batch = np.zeros((32, 28, 28))
7 flat_batch = batch.reshape((32,
8                               784))
9
10 # Transpose
11 W = np.zeros((784, 128))
12 W_T = W.T # shape: (128, 784)
13
14 # Add batch dimension
15 x = np.zeros((784,))
16 x_batch = x.reshape((1, 784))
```

Neural network layers apply geometric transformations to input data:

- **Linear (affine):** rotation, scaling, translation
- **Activation:** nonlinear warping/folding

A deep network learns to *unfold* and *flatten* tangled data manifolds until classes become linearly separable.

2D analogy:

Two interleaved spirals (not linearly separable)

↓ nonlinear transformations

Two separated clusters (linearly separable)

Each layer stretches, rotates, and warps space a little. Many layers compose into a powerful overall transformation.



Table of Contents

4 The Keras/TensorFlow Ecosystem

- ▶ Recap and Roadmap
- ▶ Tensors: The Data Structure of Deep Learning
- ▶ Tensor Operations
- ▶ **The Keras/TensorFlow Ecosystem**
- ▶ Building an MLP: Architecture Decisions
- ▶ MNIST End-to-End
- ▶ Architecture Design Choices
- ▶ Experiments and Diagnostics

Why use a framework?

- Automatic differentiation (backprop)
- GPU/TPU acceleration
- Pre-built layers and optimizers
- Data pipelines and preprocessing
- Model serialization and deployment

Major frameworks (2024+):

- **TensorFlow/Keras:**
Production-ready, wide ecosystem
- **PyTorch:** Research standard, dynamic graphs
- **JAX:** Functional, XLA compilation

The core concepts (layers, losses, optimizers) are the same across all frameworks.

Keras provides a user-friendly interface for building and training neural networks.

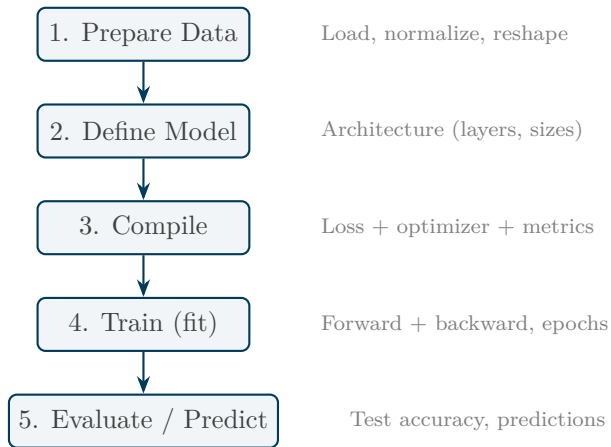
Core abstractions:

- **Layer:** the fundamental building block
- **Model:** a graph of layers
- **Optimizer:** gradient update rule
- **Loss:** objective function
- **Metric:** evaluation measure

```
1 import tensorflow as tf
2 from tensorflow import keras
3 from keras import layers
4
5 # Check version
6 print(tf.__version__)
7 # 2.x or keras 3.x
8
9 # Key submodules:
10 # keras.layers
11 # keras.optimizers
12 # keras.losses
13 # keras.metrics
14 # keras.callbacks
```

The Keras Workflow (5 Steps)

4 The Keras/TensorFlow Ecosystem



Tasks:

- Load raw data
- Normalize inputs to $[0, 1]$ or standardize to $\mu = 0, \sigma = 1$
- Reshape to expected format
- Encode labels (one-hot or integer)
- Split into train/validation/test

```
1 from keras.datasets import mnist
2 from keras.utils import
   to_categorical
3
4 # Load
5 (X_train, y_train), (X_test, y_test)
   = \
6     mnist.load_data()
7
8 # Reshape and normalize
9 X_train = X_train.reshape(
10     (60000, 784)).astype("float32") /
   255
11 X_test = X_test.reshape(
12     (10000, 784)).astype("float32") /
   255
13
14 # One-hot encode labels
15 y_train = to_categorical(y_train, 10)
16 y_test = to_categorical(y_test, 10)
```

Sequential API:

Stack layers linearly (most common for simple architectures).

Functional API:

For multi-input, multi-output, or branching architectures.

Each Dense layer specifies:

- Number of units (output dim)
- Activation function

```
1 from keras import Sequential
2 from keras.layers import Dense
3
4 model = Sequential([
5     Dense(128, activation="relu",
6         input_shape=(784,)),
7     Dense(64, activation="relu"),
8     Dense(10, activation="softmax")
9 ])
10
11 model.summary()
12 # Total params: 109,386
13 # (784*128+128)+(128*64+64)
   + (64*10+10)
```

Compilation configures the training process.

Three required components:

- **Optimizer:** how weights are updated (SGD, Adam, RMSprop)
- **Loss function:** what we minimize (cross-entropy, MSE)
- **Metrics:** what we monitor (accuracy, AUC)

```
1 model.compile(  
2     optimizer="adam",  
3     loss="categorical_crossentropy",  
4     metrics=["accuracy"]  
5 )  
6  
7 # Equivalently, with explicit objects  
8 :  
9 from keras.optimizers import Adam  
10 model.compile(  
11     optimizer=Adam(learning_rate  
12         =0.001),  
13     loss="categorical_crossentropy",  
14     metrics=["accuracy"]  
15 )
```

`model.fit()` runs the training loop:

1. Sample a batch of data
2. Forward pass (compute predictions)
3. Compute loss
4. Backward pass (compute gradients)
5. Update weights
6. Repeat for all batches (one epoch)

Key hyperparameters:

`epochs`, `batch_size`, `validation_split`

```
1 history = model.fit(  
2     X_train, y_train,  
3     epochs=10,  
4     batch_size=128,  
5     validation_split=0.15  
6 )  
7  
8 # history.history contains:  
9 # - "loss", "accuracy"  
10 # - "val_loss", "val_accuracy"  
11 # for each epoch
```

Evaluate on test set:

Reports loss and metrics on unseen data.

Generate predictions:

Returns probability vectors (softmax output) or class labels.

The test set must *never* be used during training or hyperparameter selection.

```
1 # Evaluate
2 test_loss, test_acc = model.evaluate(
3     X_test, y_test)
4 print(f"Test accuracy: {test_acc:.4f}"
5       ")
6
7 # Predict
8 predictions = model.predict(X_test
9                             [:5])
10 print(predictions[0])
11
12 # Get class labels
13 import numpy as np
14 labels = np.argmax(predictions, axis
15                     =1)
```



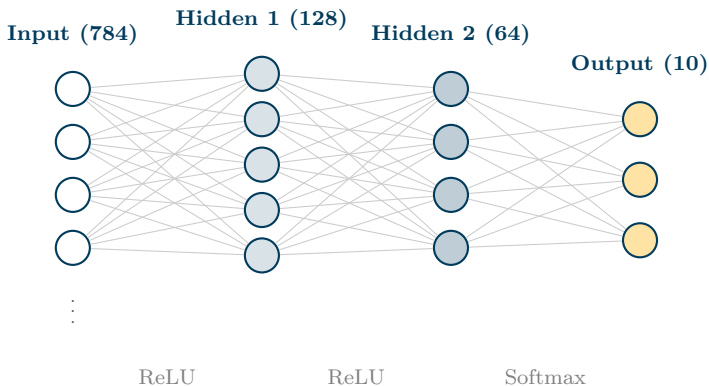

Table of Contents

5 Building an MLP: Architecture Decisions

- ▶ Recap and Roadmap
- ▶ Tensors: The Data Structure of Deep Learning
- ▶ Tensor Operations
- ▶ The Keras/TensorFlow Ecosystem
- ▶ **Building an MLP: Architecture Decisions**
- ▶ MNIST End-to-End
- ▶ Architecture Design Choices
- ▶ Experiments and Diagnostics

Multilayer Perceptron (MLP) Architecture

5 Building an MLP: Architecture Decisions



Fully connected: every neuron connects to all neurons in the adjacent layers.

Total parameters: $(784 \times 128 + 128) + (128 \times 64 + 64) + (64 \times 10 + 10) = 109,386$

Dense (Fully Connected) Layer:

- Input: vector of size d_{in}
- Output: vector of size d_{out}
- Parameters: $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$
- Computation: $\mathbf{a} = \sigma(W^T \mathbf{x} + \mathbf{b})$

A Dense layer with n units has $d_{\text{in}} \times n + n$ learnable parameters.

```
1 # Single dense layer
2 layer = Dense(128, activation="relu")
3
4 # Parameters breakdown:
5 # If input_dim = 784:
6 # Weights: 784 * 128 = 100,352
7 # Biases: 128
8 # Total: 100,480
9
10 # Access weights after build:
11 W, b = layer.get_weights()
12 print(W.shape) # (784, 128)
13 print(b.shape) # (128,)
```

Hidden layers:

- **ReLU:** $f(x) = \max(0, x)$
 - Default choice for hidden layers
 - Mitigates vanishing gradient
 - Computationally efficient

Output layer (depends on task):

- **Softmax:** multi-class classification
- **Sigmoid:** binary classification
- **Linear (none):** regression

Softmax:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Converts raw logits into a probability distribution. All outputs sum to 1.

```
1 # Output layer for 10 classes
2 Dense(10, activation="softmax")
3
4 # Binary classification
5 Dense(1, activation="sigmoid")
6
7 # Regression
8 Dense(1, activation=None)
```

Task	Output Activation	Loss Function
Multi-class (one-hot labels)	Softmax	<code>categorical_crossentropy</code>
Multi-class (integer labels)	Softmax	<code>sparse_categorical_crossentropy</code>
Binary classification	Sigmoid	<code>binary_crossentropy</code>
Regression	None (linear)	<code>mse</code> (mean squared error)

Categorical cross-entropy:

$$\mathcal{L} = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

where y_i is the true one-hot label and $\hat{y}_i$ is the predicted probability.

For a correct class c : $\mathcal{L} = -\log(\hat{y}_c)$. Lower confidence $\hat{y}_c \rightarrow 0$ means higher loss.

Stochastic Gradient Descent (SGD):

$$W \leftarrow W - \eta \nabla_W \mathcal{L}$$

Simple, but sensitive to learning rate and can oscillate.

Adam (Kingma and Ba, 2015):

- Adaptive per-parameter learning rates
- Maintains running mean and variance of gradients
- Default choice for most tasks
- Typical $\eta = 0.001$

```
1 # SGD with momentum
2 from keras.optimizers import SGD
3 opt = SGD(learning_rate=0.01,
4           momentum=0.9)
5
6 # Adam (recommended default)
7 from keras.optimizers import Adam
8 opt = Adam(learning_rate=0.001)
9
10 # RMSprop
11 from keras.optimizers import
12     RMSprop
13 opt = RMSprop(learning_rate=0.001)
14 model.compile(optimizer=opt, ...)
```

Adam maintains two running estimates per parameter:

1. **First moment** m_t (mean of gradients)
2. **Second moment** v_t (mean of squared gradients)

Update rules:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

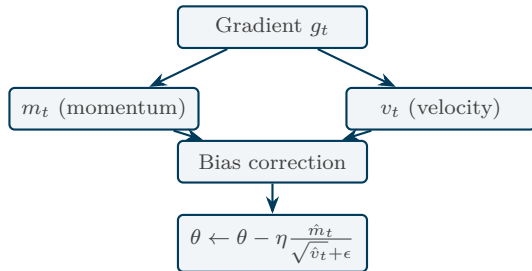
$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Default hyperparameters:

$$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-7}$$



Definitions:

- **Epoch:** one pass through the entire training set
- **Batch:** subset of samples processed together
- **Iteration:** one weight update (one batch)

Example:

60,000 samples, batch size 128:

$\lceil 60000/128 \rceil = 469$ iterations per epoch.

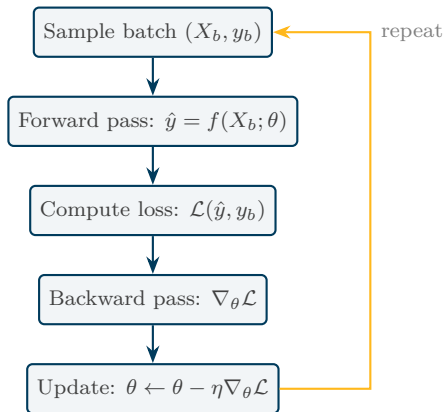
10 epochs = 4,690 weight updates.

Batch size trade-offs:

Small batch	Large batch
Noisy gradients	Stable gradients
Better generalization	Faster per epoch
More updates/epoch	Fewer updates/epoch
Low GPU utilization	High GPU utilization

Common choices: 32, 64, 128, 256.

Must fit in GPU memory.



Keras handles this loop internally. You control it via `epochs`, `batch_size`, and `callbacks`.

Training loss always decreases (if learning rate is correct).

Validation loss reveals overfitting:

- If val_loss decreases: model is generalizing
- If val_loss increases while train_loss decreases: overfitting

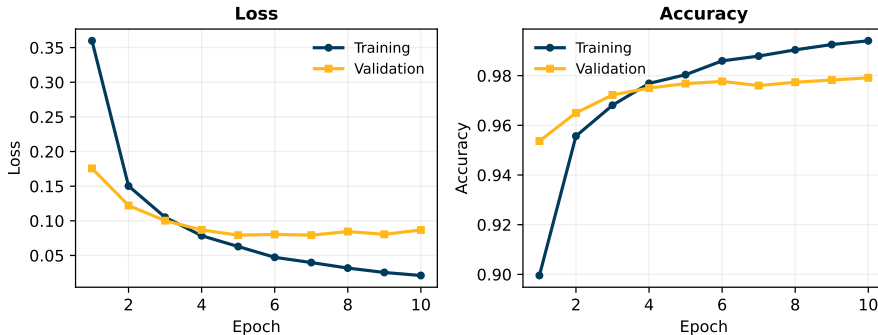
Best practice:

Always set `validation_split` or provide a separate validation set.

```
1 # Option 1: automatic split
2 history = model.fit(
3     X_train, y_train,
4     validation_split=0.15,
5     epochs=20,
6     batch_size=128
7 )
8
9 # Option 2: explicit validation set
10 history = model.fit(
11     X_train, y_train,
12     validation_data=(X_val, y_val),
13     epochs=20,
14     batch_size=128
15 )
```

```
1 import matplotlib.pyplot as plt
2
3 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
4
5 # Loss
6 ax1.plot(history.history["loss"], label="Train")
7 ax1.plot(history.history["val_loss"], label="Validation")
8 ax1.set_xlabel("Epoch"); ax1.set_ylabel("Loss")
9 ax1.legend()
10
11 # Accuracy
12 ax2.plot(history.history["accuracy"], label="Train")
13 ax2.plot(history.history["val_accuracy"], label="Validation")
14 ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy")
15 ax2.legend()
16 plt.tight_layout(); plt.show()
```

Tracking loss and accuracy curves is the primary diagnostic tool for identifying underfitting, overfitting, and convergence issues.



Left: Loss decreases for both train and validation. A widening gap signals overfitting.
Right: Accuracy improves rapidly in early epochs, then plateaus. The validation curve is the true measure of generalization.



Table of Contents

6 MNIST End-to-End

- ▶ Recap and Roadmap
- ▶ Tensors: The Data Structure of Deep Learning
- ▶ Tensor Operations
- ▶ The Keras/TensorFlow Ecosystem
- ▶ Building an MLP: Architecture Decisions
- ▶ **MNIST End-to-End**
- ▶ Architecture Design Choices
- ▶ Experiments and Diagnostics

Dataset:

- 70,000 handwritten digit images (0–9)
- 60,000 train / 10,000 test
- 28×28 grayscale pixels
- 10 balanced classes

Why MNIST?

- Small enough to train on CPU
- Well-understood benchmarks
- Non-trivial (requires learning features)
- Built into Keras

Sample digits:



Task:

Given a 28×28 image, predict which digit (0–9) it represents.

We will build a classifier for this dataset today.

```
1 import numpy as np
2 from tensorflow import keras
3 from keras.datasets import mnist
4 from keras.utils import to_categorical
5
6 # 1. Load data
7 (X_train, y_train), (X_test, y_test) = mnist.load_data()
8
9 # 2. Preprocess
10 X_train = X_train.reshape((60000, 784)).astype("float32") / 255
11 X_test  = X_test.reshape((10000, 784)).astype("float32") / 255
12
13 # 3. One-hot encode labels
14 y_train = to_categorical(y_train, 10)
15 y_test  = to_categorical(y_test, 10)
16
17 print(f"Train: {X_train.shape}, Test: {X_test.shape}")
18 # Train: (60000, 784), Test: (10000, 784)
```

```
1 from keras import Sequential
2 from keras.layers import Dense
3
4 # 4. Define architecture
5 model = Sequential([
6     Dense(128, activation="relu", input_shape=(784,)),
7     Dense(64, activation="relu"),
8     Dense(10, activation="softmax")
9 ])
10
11 # 5. Compile
12 model.compile(
13     optimizer="adam",
14     loss="categorical_crossentropy",
15     metrics=["accuracy"]
16 )
17
18 model.summary()
```

Parameters: $100,480 + 8,256 + 650 = \mathbf{109,386}$ trainable weights.

Complete MNIST Code (Part 3: Train and Evaluate)

6 MNIST End-to-End

```
1 # 6. Train
2 history = model.fit(
3     X_train, y_train,
4     epochs=10,
5     batch_size=128,
6     validation_split=0.15,
7     verbose=1
8 )
9
10 # 7. Evaluate on test set
11 test_loss, test_acc = model.evaluate(X_test, y_test)
12 print(f"Test accuracy: {test_acc:.4f}")
13
14 # 8. Predict
15 predictions = model.predict(X_test[:5])
16 predicted_labels = np.argmax(predictions, axis=1)
17 print(f"Predicted: {predicted_labels}")
```

```
1 Model: "sequential"
2
3 -----
4 Layer (type)                Output Shape                Param #
5 -----
6 dense (Dense)                (None, 128)                 100480
7 dense_1 (Dense)              (None, 64)                  8256
8 dense_2 (Dense)              (None, 10)                  650
9 -----
10 Total params: 109,386
11 Trainable params: 109,386
12 Non-trainable params: 0
13 -----
```

- None in Output Shape = batch dimension (determined at runtime)
- Param # = $d_{\text{in}} \times d_{\text{out}} + d_{\text{out}}$
- All parameters are trainable here (no frozen layers)

```
1 Epoch 1/10
2 399/399 [=====] - 2s 4ms/step - loss: 0.3521 - accuracy: 0.8987 - val_loss: 0.1612 - val_accuracy:
   0.9527
3 Epoch 2/10
4 399/399 [=====] - 1s 3ms/step - loss: 0.1453 - accuracy: 0.9573 - val_loss: 0.1198 - val_accuracy:
   0.9649
5 Epoch 3/10
6 399/399 [=====] - 1s 3ms/step - loss: 0.0987 - accuracy: 0.9707 - val_loss: 0.1024 - val_accuracy:
   0.9702
7 ...
8 Epoch 10/10
9 399/399 [=====] - 1s 3ms/step - loss: 0.0198 - accuracy: 0.9943 - val_loss: 0.0896 - val_accuracy:
   0.9782
```

- **399 steps:** $\lceil 51000/128 \rceil = 399$ (85% of 60K after val split)
- **Train accuracy** reaches 99.4% but **val accuracy** plateaus at 97.8%
- Gap indicates mild overfitting (addressed in subsequent lectures)

Layer 1 (784 $\rightarrow$ 128):

Detects local patterns: edges, strokes, curves.
Each of 128 units specializes in a different pattern.

Layer 2 (128 $\rightarrow$ 64):

Combines patterns into parts: loops, vertical lines, crossings. More abstract features.

Layer 3 (64 $\rightarrow$ 10):

Maps from parts to digit identities.
Essentially a linear classifier on learned features.

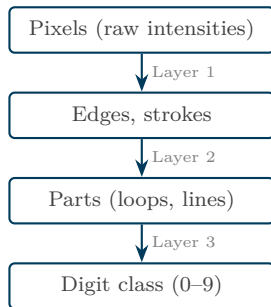
Hierarchy of representations:



Table of Contents

7 Architecture Design Choices

- ▶ Recap and Roadmap
- ▶ Tensors: The Data Structure of Deep Learning
- ▶ Tensor Operations
- ▶ The Keras/TensorFlow Ecosystem
- ▶ Building an MLP: Architecture Decisions
- ▶ MNIST End-to-End
- ▶ **Architecture Design Choices**
- ▶ Experiments and Diagnostics

More layers = more abstraction.

But diminishing returns and training difficulty:

- 1 hidden layer: universal approximator (in theory)
- 2–3 hidden layers: sufficient for most tabular tasks
- 5+ layers: diminishing returns without skip connections

Rule of thumb for MLPs:

Start with 2 hidden layers. Add more only if underfitting persists.

```
1 # Shallow: 1 hidden layer
2 model_shallow = Sequential([
3     Dense(256, activation="relu",
4         input_shape=(784,)),
5     Dense(10, activation="softmax")
6 ])
7
8 # Deeper: 3 hidden layers
9 model_deep = Sequential([
10    Dense(256, activation="relu",
11        input_shape=(784,)),
12    Dense(128, activation="relu"),
13    Dense(64, activation="relu"),
14    Dense(10, activation="softmax")
15 ])
```

Common patterns:

- **Funnel:** decreasing width ($512 \rightarrow 256 \rightarrow 128$)
- **Constant:** same width throughout ($256 \rightarrow 256 \rightarrow 256$)
- **Bottleneck:** narrow middle layer

Guidelines:

- Output dimension should not be a bottleneck (never smaller than the number of classes)
- Powers of 2 are efficient on GPU (32, 64, 128, 256, 512)
- Wider layers = more capacity but more parameters

```
1 # Funnel (most common for MLP)
2 model = Sequential([
3     Dense(512, activation="relu",
4         input_shape=(784,)),
5     Dense(256, activation="relu"),
6     Dense(128, activation="relu"),
7     Dense(10, activation="softmax")
8 ])
9 # Params: 784*512+512 + 512*256+256
10 #          + 256*128+128 + 128*10+10
11 #          = 567,434
12
13 # vs. single wide layer:
14 # Dense(512): 784*512+512 = 401,920
15 # Dense(10): 512*10+10 = 5,130
16 # Total: 407,050
```

Universal Approximation Theorem:

A single hidden layer with enough units can approximate any continuous function (Hornik et al., 1989).

But:

- “Enough units” may be exponentially many
- Deep networks are exponentially more efficient
- Approximation $\neq$ learnability via gradient descent
- Generalization requires additional constraints (regularization)

The capacity trade-off:

Too few params	Too many params
Underfitting	Overfitting
High bias	High variance
Can't learn	Memorizes noise

The goal: find the minimum model size

that achieves low training loss, then regularize to close the train/val gap.

Initial weights determine the starting point of optimization. Bad initialization can prevent learning.

Key principles:

- Do not initialize all weights to zero (symmetry breaking)
- Do not initialize too large (exploding activations)
- Do not initialize too small (vanishing signal)

Keras defaults:

Dense layers use Glorot uniform by default. Switch to He initialization for ReLU.

Glorot/Xavier (sigmoid/tanh):

$$W \sim \mathcal{U} \left[-\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}} \right]$$

He (ReLU):

$$W \sim \mathcal{N} \left(0, \frac{2}{n_{\text{in}}} \right)$$

```
1 Dense(128, activation="relu",  
2     kernel_initializer="he_normal"  
    )
```

Without bias, the hyperplane must pass through the origin.

With bias:

$$z = W\mathbf{x} + \mathbf{b}$$

The bias shifts the activation function, allowing the neuron to activate even when all inputs are zero.

Initialization:

Biases are typically initialized to zero. For ReLU, a small positive value (0.01) can help avoid dead neurons.

```
1 # Default: bias initialized to
   zeros
2 Dense(128, activation="relu",
3       use_bias=True,
4       bias_initializer="zeros")
5
6 # No bias (rare, used in BatchNorm)
7 Dense(128, activation="relu",
8       use_bias=False)
9
10 # Count parameters:
11 # With bias: 784*128 + 128 =
   100,480
12 # No bias: 784*128 =
   100,352
```

Neural networks work best when input features are:

- Small values (typically in $[0, 1]$ or $[-1, 1]$)
- Approximately homogeneous (similar scales)
- Centered around zero (for standardization)

Normalization: $x' = x/x_{\max}$

Standardization: $x' = (x - \mu)/\sigma$

Always compute μ and σ from training data only. Apply the same transform to validation/test.

```
1 # Normalization (for pixel data)
2 X_train = X_train / 255.0
3 X_test  = X_test  / 255.0
4
5 # Standardization (for tabular data)
6 mean = X_train.mean(axis=0)
7 std  = X_train.std(axis=0)
8 X_train = (X_train - mean) / std
9 X_test  = (X_test - mean) / std
10 # Note: use TRAIN stats for test
11
12 # Why? Large input values cause
13 # large activations, unstable
14 # gradients, slow convergence.
```

Classification labels must be encoded as tensors.

Integer encoding:

Label “7” stored as integer 7.

Use with

`sparse_categorical_crossentropy`.

One-hot encoding:

Label “7” $\rightarrow$ `[0,0,0,0,0,0,0,1,0,0]`

Use with `categorical_crossentropy`.

Both are equivalent mathematically. One-hot is explicit; sparse is memory-efficient for many classes.

```
1 from keras.utils import
   to_categorical
2
3 y_train_int = np.array([7, 2, 1, 0,
   4                          4])
5
6 # One-hot encode
7 y_train_oh = to_categorical(
8     y_train_int, num_classes=10)
9 print(y_train_oh[0])
10 # [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
11
12 # Alternative: use sparse loss
13 model.compile(
14     loss="
15         sparse_categorical_crossentropy
16         ",
17     ... # no need to one-hot encode
18 )
```



Table of Contents

8 Experiments and Diagnostics

- ▶ Recap and Roadmap
- ▶ Tensors: The Data Structure of Deep Learning
- ▶ Tensor Operations
- ▶ The Keras/TensorFlow Ecosystem
- ▶ Building an MLP: Architecture Decisions
- ▶ MNIST End-to-End
- ▶ Architecture Design Choices
- ▶ Experiments and Diagnostics

Width experiment:

Fix depth at 1 hidden layer. Vary units: 16, 64, 256, 1024.

What to observe:

- Accuracy improves with more units
- Diminishing returns at some point
- More units = more parameters = slower

Depth experiment:

Fix width at 128 units per layer. Vary layers: 1, 2, 3, 5.

What to observe:

- 2 layers often outperform 1
- Beyond 2–3, returns diminish for simple tasks
- Very deep MLPs can be harder to optimize

Run the accompanying notebook (Section 7) to see actual results on MNIST.

The learning rate η is the most important hyperparameter.

η	Behavior
0.1	Diverges or oscillates
0.01	Converges, slightly unstable
0.001	Smooth convergence (default)
0.0001	Very slow convergence

Too high: overshoots minima.

Too low: takes too many epochs.

```
1 from keras.optimizers import Adam
2
3 for lr in [0.1, 0.01, 0.001,
4           0.0001]:
5     model = Sequential([
6         Dense(128, activation="relu",
7             input_shape=(784,)),
8         Dense(10, activation="softmax")
9     ])
10    model.compile(
11        optimizer=Adam(
12            learning_rate=lr),
13        loss="categorical_crossentropy",
14        metrics=["accuracy"])
15    model.fit(X_train, y_train,
16            epochs=10, verbose=0)
```

Batch size affects both speed and generalization.

Smaller batches (32, 64):

- Noisier gradient estimates
- Acts as implicit regularization
- More weight updates per epoch
- Slower wall-clock time per epoch

Larger batches (512, 2048):

- Smoother, more stable gradients
- Better GPU utilization
- Fewer updates per epoch
- May require learning rate scaling

Common choices: 32, 64, 128, 256.

Must fit in GPU memory.

Try different batch sizes in the notebook and compare convergence speed.

Shape errors:

- Forgetting to flatten images
- Mismatch between `input_shape` and actual data
- Wrong number of output units

Loss/activation mismatch:

- Using `categorical_crossentropy` with integer labels
- Sigmoid output with softmax loss
- No activation on hidden layers

Training issues:

- Not normalizing inputs (training is very slow)
- Learning rate too high (loss = NaN)
- Not shuffling data (biased batches)
- Evaluating on training set (false confidence)

Debugging tip:

If loss does not decrease after epoch 1, check data preprocessing and learning rate first.

Before training:

1. Verify input shapes match model expectations
2. Check label encoding matches loss function
3. Verify model parameter count is reasonable
4. Test forward pass with one batch

```
1 # 1. Check shapes
2 print(X_train.shape) # (60000, 784)
3 print(y_train.shape) # (60000, 10)
4
5 # 2. Verify labels
6 print(y_train[0]) # one-hot vector
7
8 # 3. Check model
9 model.summary()
10
11 # 4. Overfit a tiny subset
12 # (should reach ~100% quickly)
13 model.fit(X_train[:100], y_train
14           [:100],
15           epochs=50, verbose=0)
16 _, acc = model.evaluate(
17     X_train[:100], y_train[:100])
18 assert acc > 0.99, "Cannot overfit!"
```

For architectures that are not purely sequential:

- Multiple inputs or outputs
- Shared layers
- Residual connections
- Branching and merging

Layers are called like functions, passing the previous layer's output as input.

Same model, same parameters. The Functional API just gives more flexibility in how layers connect.

```
1 from keras import Input, Model
2 from keras.layers import Dense
3
4 # Same MLP as before,
5 # using the Functional API
6 inputs = Input(shape=(784,))
7 x = Dense(128, activation="relu")(
8     inputs)
9 x = Dense(64, activation="relu")(x)
10 outputs = Dense(10,
11     activation="softmax")(x)
12
13 model = Model(inputs=inputs,
14     outputs=outputs)
15 model.summary()
```

Why save models?

- Resume training after interruption
- Deploy trained model for inference
- Share with collaborators
- Version control experiments

Formats:

- `.keras`: recommended (Keras 3)
- `.h5`: legacy HDF5 format
- `SavedModel`: TensorFlow serving

```
1  # Save entire model
2  model.save("mnist_mlp.keras")
3
4  # Load model
5  from keras.models import load_model
6  loaded = load_model("mnist_mlp.keras"
7                      )
8
9  # Save only weights
10 model.save_weights("mnist_weights.h5"
11                    )
12
13 # Load weights into same architecture
14 new_model = build_model()
15 new_model.load_weights("mnist_weights
16                        .h5")
17
18 # Evaluate loaded model
19 loaded.evaluate(X_test, y_test)
```

MNIST is a starting point. Real-world tasks differ:

- Higher resolution images ($224 \times 224 \times 3$)
- More classes (ImageNet: 1000)
- Class imbalance
- Noisy labels
- Distribution shift between train and deploy

MLPs struggle with high-dimensional spatial data. Subsequent lectures cover CNNs (spatial patterns) and Transformers (sequential patterns).

When MLPs are appropriate:

- Tabular data (structured features)
- Low-dimensional inputs
- Classification/regression after feature extraction
- Final layers of larger architectures

When MLPs are not enough:

- Images (use CNNs)
- Text/sequences (use Transformers)
- Graphs (use GNNs)

1. **Tensors** are the fundamental data structure: rank-0 (scalar) through rank-5 (video). Every operation in DL is a tensor operation.
2. **Element-wise ops, dot products, broadcasting, and reshaping** are the four core tensor operations that compose into neural network layers.
3. **A Dense layer** computes $\sigma(Wx + b)$ with $d_{\text{in}} \times d_{\text{out}} + d_{\text{out}}$ parameters.
4. **The Keras workflow** is five steps: prepare data, define model, compile, train, evaluate.
5. **MNIST with a 2-layer MLP** achieves $\sim 98\%$ accuracy with 109K parameters and under 20 lines of code.
6. **Architecture choices** (depth, width, activations, learning rate) must be tuned empirically. Start simple.
7. **Preprocessing matters:** normalize inputs, encode labels correctly, always validate on held-out data.